



VIT[®]
B H O P A L
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh – 466114

CSA1021 INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING

REG.NO: 26BHI10069

NAME: MOHAMMED NIFAL V

BRANCH: HEALTH INFORMATICS

REPOSITORY LINK: <https://github.com/mohammednifal26bhi10069/VITyarthi-project>

Detailed Project Report Python Console-Based Application

Project Type	Menu-driven hospital management application
Implementation	Python 3
Data Storage	In-memory lists and dictionaries
Interface	Console / command-line
Report Sections	12 required submission sections

Prepared from the supplied Hospital Patient Management System source program.

1. Introduction

The Hospital Patient Management System (HPMS) is a menu-driven Python application developed to manage common hospital administration tasks from a command-line interface. The program organizes patient information, medical records, doctor details, appointments, laboratory tests, medicines, billing information, and patient discharge within a single script.

The application uses Python lists to hold collections of patients, doctors, appointments, laboratory tests, and medicines. Individual records are represented with dictionaries containing the fields needed by each feature. A continuous main-menu loop connects the individual functions and allows the user to perform multiple operations during one program session.

The system is intentionally lightweight: it does not depend on external libraries, a database, a web server, or a graphical user interface. This makes it suitable as an academic project demonstrating modular programming, data structures, input validation, record searching, appointment handling, and calculation of hospital charges.

2. Problem Statement

Manual handling of hospital information can make it difficult to keep patient details, medical information, appointments, laboratory records, medicine charges, and bills organized. A basic computerized system is therefore needed to provide a structured way to enter, search, display, update, and calculate this information.

The project addresses this need by providing a single console application with separate functions for the major hospital-management activities. The system also validates several numeric inputs, prevents duplicate patient and doctor identifiers/names, blocks selected operations for discharged patients, and automatically recalculates bill components when laboratory tests or medicines are added.

3. Functional Requirements

ID	Requirement	Description
FR-01	Patient registration	The system shall accept and store patient ID, name, gender, phone, address, age, disease, and doctor name
FR-02	Patient display/search	The system shall display patient records and search patients by ID or name.
FR-03	Medical records	The system shall store diagnosis, symptoms, prescription, test results, and treatment details and allow those
FR-04	Doctor management	The system shall add doctor information, department, and consultation fee and display doctor details.
FR-05	Appointments	The system shall book, view, and cancel appointments for registered patients and available doctors.
FR-06	Laboratory tests	The system shall add and display laboratory test name, fee, and result for a patient.
FR-07	Medicine records	The system shall add and display medicine name, quantity, price, and calculated total charge.
FR-08	Billing	The system shall calculate and display doctor, laboratory, pharmacy/medicine, administrative, other, and total
FR-09	Discharge	The system shall display final bill information and change a patient's status to discharged.
FR-10	Menu control	The system shall provide a numbered menu and an exit option.

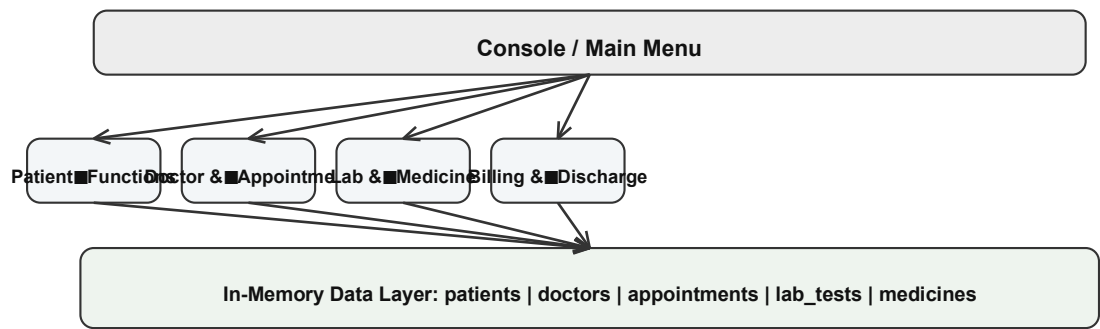
4. Non-functional Requirements

Quality	Requirement
Usability	The numbered console menu should be straightforward to operate and each operation provides prompts and success
Input validation	Numeric values such as fees, quantities, and prices are converted and checked using exception handling in the relevant
Reliability	The program checks for missing patients/doctors/appointments in several operations and handles duplicate patient IDs
Maintainability	Hospital operations are separated into functions such as <code>add_patient()</code> , <code>book_appointment()</code> , <code>generate_bill()</code> , <code>add_lab_bill()</code>
Portability	The program uses standard Python functionality and does not require external packages.
Performance	For the small in-memory datasets expected by this academic project, linear searches through Python lists are sufficient
Data persistence	Persistent storage is not a requirement of the current implementation; data exists only for the duration of the programs

Security	Authentication, authorization, encryption, and database-level access control are not implemented in the
----------	---

5.System Architecture

The application follows a simple single-process, function-oriented architecture. The main menu receives user commands and dispatches them to specialized functions. Those functions read from or update five in-memory collections: patients, doctors, appointments, lab_tests, and medicines.



Single Python script; functions operate directly on lists of dictionaries.

Figure 1. High-level component architecture.

There is no separate database or service layer. The program starts with empty lists, modifies them during execution, and exits through the main menu. Consequently, all records are lost when the program terminates.

6.Design Diagrams

6.1Use Case Diagram

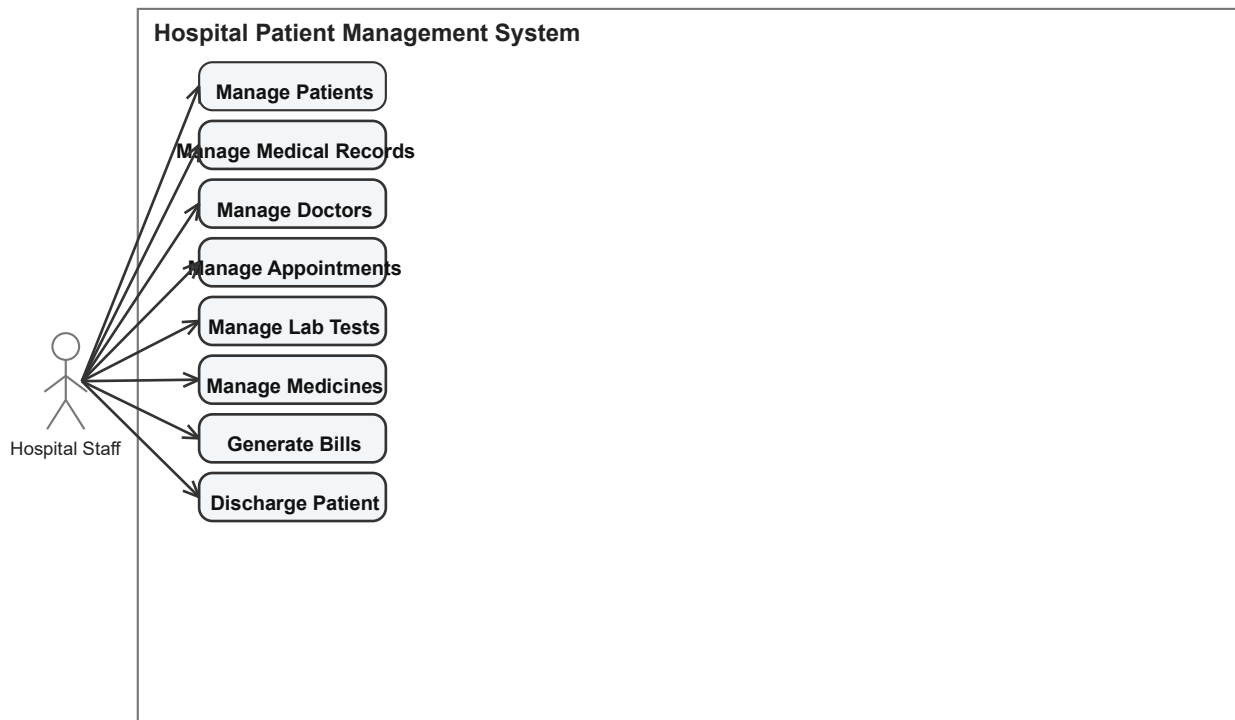


Figure 2. Use cases supported by the hospital staff/user actor.

6.2Workflow Diagram

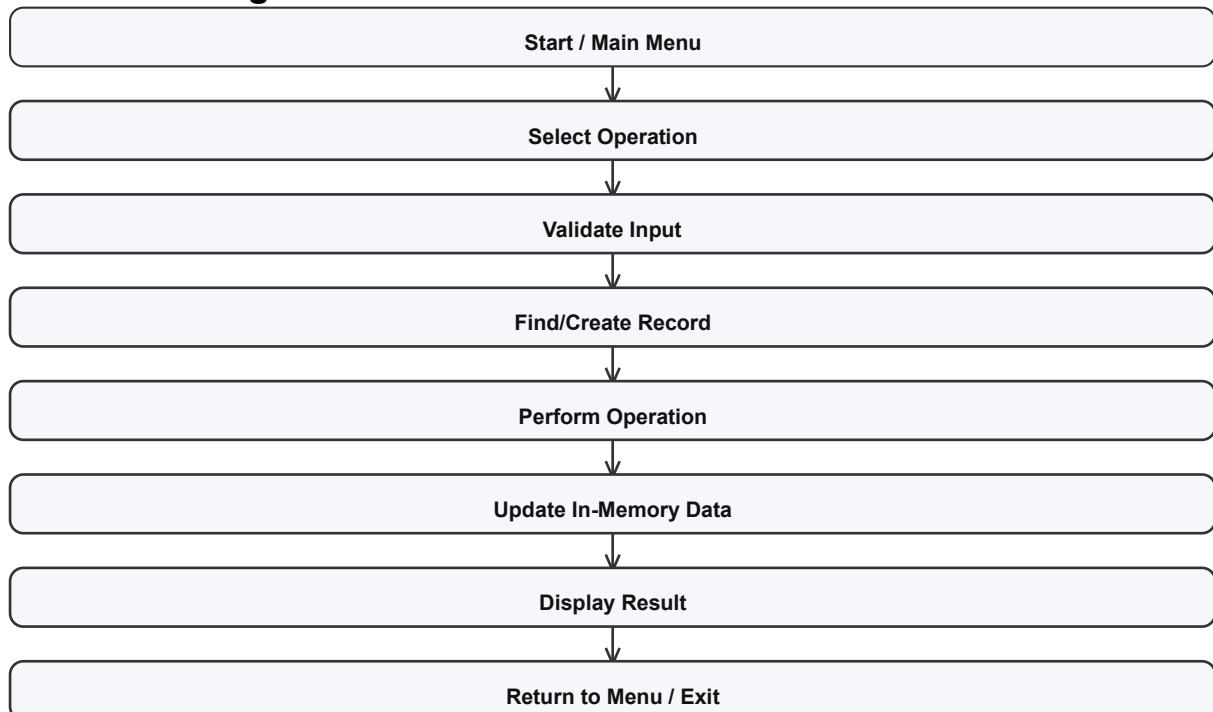


Figure 3. General workflow for menu-driven operations.

6.3 Sequence Diagram

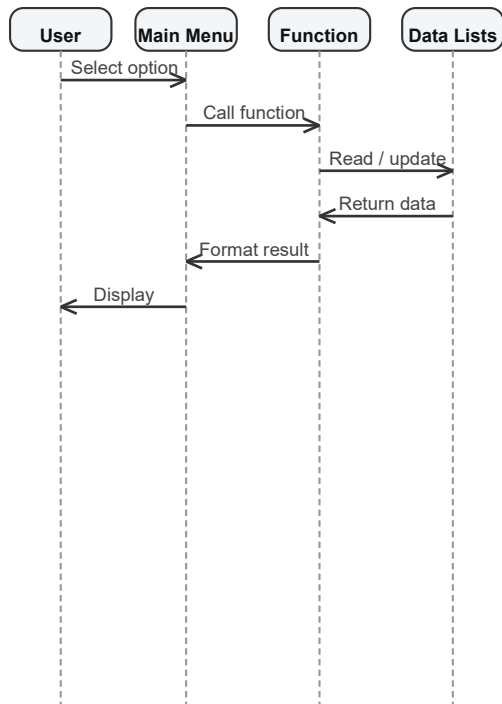
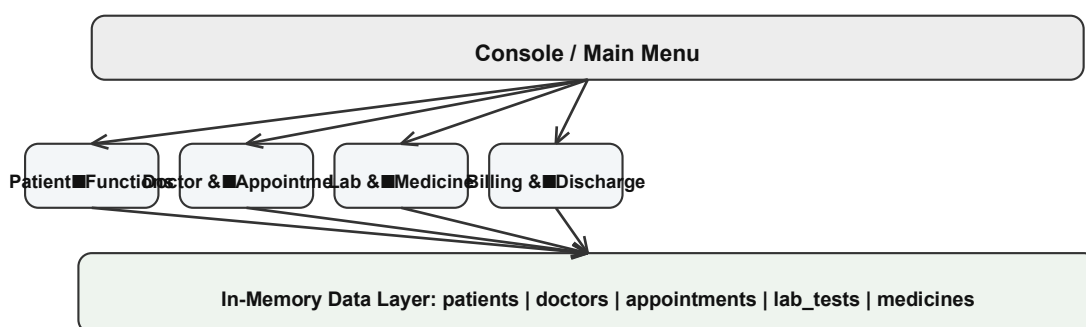


Figure 4. Generic sequence for a user-selected operation.

The exact function called depends on the selected menu option. For example, selecting Generate Bill invokes `generate_bill()`, while adding a laboratory test invokes `add_lab_test()`. The function accesses the relevant list(s), updates records, and prints the result before control returns to the menu loop.

6.4 Class / Component Diagram

The supplied program does not define Python classes. It is function-oriented and uses dictionaries for records. Therefore, a component diagram is more accurate than a traditional UML class diagram for the current implementation.

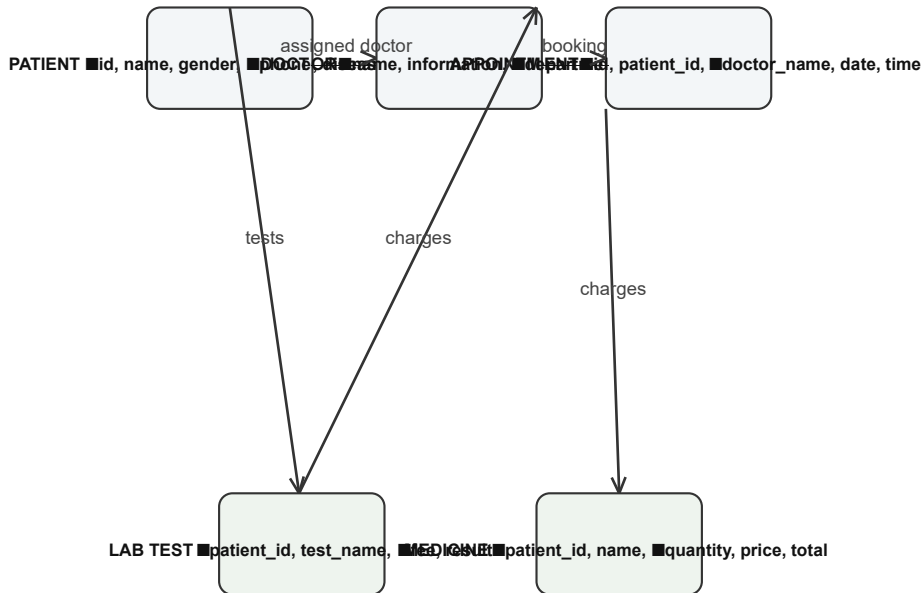


Single Python script; functions operate directly on lists of dictionaries.

Figure 5. Component view of the function-oriented implementation.

6.5ER Diagram / Logical Data Model

Although the project does not use a persistent database, it does use structured in-memory data collections. The following ER-style logical model documents the relationships represented by the dictionaries and their identifiers.



Logical in-memory data model; no persistent database is implemented.

Figure 6. Logical in-memory data model (ER-style representation)

7.Design Decisions & Rationale

Decision	Rationale
Lists and dictionaries instead of a database	The project is an academic console application and the supplied implementation keeps records in lists of dictionaries
Function-oriented structure	Each major operation is implemented as a separate function. This makes the program easier to follow
Single main menu	continuous while-loop provides one entry point for all operations and makes the application easy to navigate
ID/name search	Patient records and medical records can be located by patient ID or name, providing two practical lookup
Automatic bill recalculation	Laboratory and medicine entries update the patient's bill through <code>recalculate_patient_bill()</code> , reducing manual
Input validation with try/except	Fees, prices, and quantities are converted to numeric types and invalid values are rejected with a user-face
Status-based discharge control	Patients are initialized as admitted and changed to discharged after the discharge operation.

8.Implementation Details

8.1Data Structures

- **patients:** list of patient dictionaries containing personal details, medical-record fields, billing fields, and status.
- **doctors:** list of dictionaries containing doctor name, information, department, and consultation fee.
- **appointments:** list of dictionaries containing appointment ID, patient details, doctor name, date, time, and status.
- **lab_tests:** list of dictionaries containing patient ID/name, test name, fee, and result.
- **medicines:** list of dictionaries containing patient ID/name, medicine name, quantity, price, and total.

8.2Main Functional Modules

Module	Functions
Patient	<code>add_patient()</code> , <code>display_patients()</code> , <code>search_patient()</code> , <code>show_patient()</code>
Medical Record	<code>add_medical_records()</code> , <code>search_medical_record()</code> , <code>display_medical_record()</code>
Doctor	<code>add_doctor_details()</code> , <code>display_doctors()</code>
Appointment	<code>book_appointment()</code> , <code>view_appointments()</code> , <code>cancel_appointment()</code>
Laboratory	<code>add_lab_test()</code> , <code>view_lab_tests()</code>
Medicine	<code>add_medicine()</code> , <code>view_medicines()</code>
Billing	<code>recalculate_patient_bill()</code> , <code>generate_bill()</code>
Discharge	<code>discharge_patient()</code>

8.3Billing Logic

The bill is recalculated by summing the patient's doctor fee, all laboratory test fees for that patient, all medicine totals for that patient, administrative fee, and other charges. The resulting value is stored in the patient's bill field. Pharmacy bill and medicine charges are both derived from the medicine totals.

```
total = doctor_fee + lab_bill + pharmacy_bill
total = total + administrative_fee +
other_charges patient["bill"] = total
```

8.4Main Menu Mapping

Option	Operation
1	ADD PATIENT
2	DISPLAY PATIENT
3	ADD MEDICAL RECORDS
4	SEARCH MEDICAL RECORD
5	SEARCH PATIENT
6	GENERATE BILL
7	ADD DOCTOR DETAILS
8	DISPLAY DOCTOR DETAILS
9	BOOK AN APPOINTMENT
10	VIEW APPOINTMENTS
11	CANCEL APPOINTMENT
12	ADD LAB TEST
13	VIEW LAB TESTS
14	ADD MEDICINE
15	VIEW MEDICINES
16	DISCHARGE PATIENT
17	EXIT

9.Screenshots / Results

```
1  # Hospital Patient Management System
2
3
4  # Lists used to store the hospital information
5  patients = []
6  doctors = []
7  appointments = []
8  lab_tests = []
9  medicines = []
10 # Add a new patient
11
12 def add_patient():
13     patient_id = input("Enter patient ID: ")
14     for patient in patients:
15         if patient["id"] == patient_id:
16             print("Patient ID already exists")
17             return
18     name = input("Enter the name: ")
19     gender = input("Enter the gender: ")
20     phone = input("Enter the phone number: ")
21     address = input("Enter the address: ")
22     age = input("Enter the age: ")
23     disease = input("Enter the disease: ")
24     doctor = input("Enter the doctor name: ")
25
26     patient = {
27         "id": patient_id,
28         "name": name,
29         "gender": gender,
30         "phone": phone,
31         "address": address,
32         "age": age,
33         "disease": disease,
34         "doctor": doctor,
35         "diagnosis": "",
36         "symptoms": "",
37         "prescription": "",
38         "test_results": "",
39         "treatment_details": "",
40         "doctor_fee": 0,
41         "administrative_fee": 0,
42         "other_charges": 0,
43         "lab_bill": 0,
44         "pharmacy_bill": 0,
45         "medicine_charges": 0,
46         "bill": 0,
47         "status": "admitted"
48     }
49     patients.append(patient)
50     print("Patient added successfully")
```

```

50 # Add doctor details
51
52 def add_doctor_details():
53     doctor_name = input("Enter doctor name: ")
54     for doctor in doctors:
55         if doctor["name"] == doctor_name:
56             print("Doctor already exists")
57             return
58     doctor_information = input("Enter doctor information: ")
59     department = input("Enter department: ")
60     consultation_fee = input("Enter consultation fee: ")
61     try:
62         consultation_fee = float(consultation_fee)
63     except ValueError:
64         print("Please enter a valid fee")
65         return
66     doctor = {
67         "name": doctor_name,
68         "information": doctor_information,
69         "department": department,
70         "consultation_fee": consultation_fee
71     }
72     doctors.append(doctor)
73     print("Doctor details added successfully")
74 # Display doctor details
75
76 def displayDoctors():
77     if len(doctors) == 0:
78         print("No doctors found")
79         return
80     print("----- DOCTOR DETAILS -----")
81     for doctor in doctors:
82         print("Doctor name:", doctor["name"])
83         print("Doctor information:", doctor["information"])
84         print("Department:", doctor["department"])
85         print("Consultation fee:", doctor["consultation_fee"])
86     print()
87
88 # Book an appointment
89
90 def book_appointment():
91     if len(patients) == 0:
92         print("No patients found")
93         return
94     if len(doctors) == 0:
95         print("No doctors found")
96         return
97     patient_id = input("Enter patient ID: ")
98     patient = None
99     for saved_patient in patients:
100         if saved_patient["id"] == patient_id:
101             patient = saved_patient
102             break
103     if patient is None:
104         print("Patient not found")
105         return
106     if patient["status"] == "discharged":
107         print("Patient is already discharged")
108         return
109     print("----- AVAILABLE DOCTORS -----")
110     for doctor in doctors:
111         print(doctor["name"])
112     doctor_name = input("Enter doctor name: ")
113     doctor = None
114     for saved_doctor in doctors:
115         if saved_doctor["name"] == doctor_name:
116             doctor = saved_doctor
117             break
118     if doctor is None:
119         print("Doctor not found")
120         return

```

```

119         return
120     appointment = {
121         "id": str(len(appointments) + 1),
122         "patient_id": patient["id"],
123         "patient_name": patient["name"],
124         "doctor_name": doctor["name"],
125         "date": input("Enter appointment date: "),
126         "time": input("Enter appointment time: "),
127         "status": "booked"
128     }
129     appointments.append(appointment)
130     print("Appointment booked successfully")
131     print("Appointment ID:", appointment["id"])
132 # View all appointments
133
134 def view_appointments():
135     if len(appointments) == 0:
136         print("No appointments found")
137         return
138     print("----- APPOINTMENTS -----")
139     for appointment in appointments:
140         print("Appointment ID:", appointment["id"])
141         print("Patient ID:", appointment["patient_id"])
142         print("Patient name:", appointment["patient_name"])
143         print("Doctor name:", appointment["doctor_name"])
144         print("Date:", appointment["date"])
145         print("Time:", appointment["time"])
146         print("Status:", appointment["status"])
147         print()
148 # Cancel an appointment
149
150 def cancel_appointment():
151     if len(appointments) == 0:
152         print("No appointments found")
153         return
154     appointment_id = input("Enter appointment ID to cancel: ")
155     for appointment in appointments:
156         if appointment["id"] == appointment_id:
157             if appointment["status"] == "cancelled":
158                 print("Appointment is already cancelled")
159             else:
160                 appointment["status"] = "cancelled"
161                 print("Appointment cancelled successfully")
162             return
163     print("Appointment not found")
164 # Display one patient's medical record
165
166 def display_medical_record(patient):
167     print("Diagnosis:", patient["diagnosis"])
168     print("Symptoms:", patient["symptoms"])
169     print("Prescription:", patient["prescription"])
170     print("Test results:", patient["test_results"])
171     print("Treatment details:", patient["treatment_details"])
172 # Display all patients
173
174 def display_patients():
175     if len(patients) == 0:
176         print("No patients found")
177         return
178     print("----- PATIENT DETAILS -----")
179     for patient in patients:
180         print("ID:", patient["id"])
181         print("Name:", patient["name"])
182         print("Gender:", patient["gender"])
183         print("Phone:", patient["phone"])
184         print("Address:", patient["address"])
185         print("Age:", patient["age"])
186         print("Disease:", patient["disease"])
187         print("Doctor name:", patient["doctor"])
188         print("Status:", patient["status"])
189         display_medical_record(patient)
190         print()
191 # Add medical records
192

```

```

C:\Users\Lenovo\Downloads> HPMS (1).py
193 def add_medical_records():
194     choice = input("Enter id or name: ")
195     if choice == "id":
196         search_value = input("Enter the ID: ")
197         for patient in patients:
198             if patient["id"] == search_value:
199                 patient["diagnosis"] = input("Enter diagnosis: ")
200                 patient["symptoms"] = input("Enter symptoms: ")
201                 patient["prescription"] = input("Enter prescription: ")
202                 patient["test_results"] = input("Enter test results: ")
203                 patient["treatment_details"] = input("Enter treatment details: ")
204                 print("Medical record added successfully")
205                 return
206         print("Patient not found")
207     elif choice == "name":
208         search_value = input("Enter the name: ")
209         for patient in patients:
210             if patient["name"] == search_value:
211                 patient["diagnosis"] = input("Enter diagnosis: ")
212                 patient["symptoms"] = input("Enter symptoms: ")
213                 patient["prescription"] = input("Enter prescription: ")
214                 patient["test_results"] = input("Enter test results: ")
215                 patient["treatment_details"] = input("Enter treatment details: ")
216                 print("Medical record added successfully")
217                 return
218         print("Patient not found")
219     else:
220         print("Invalid choice. Enter id or name")
221 # Search for a medical record
222
223 def search_medical_record():
224     choice = input("Enter id or name: ")
225     if choice == "id":
226         search_value = input("Enter patient ID: ")
227         for patient in patients:
228             if patient["id"] == search_value:
229                 print("ID:", patient["id"])
230                 print("Name:", patient["name"])
231                 display_medical_record(patient)
232                 return

```

```

C:\Users\Lenovo\Downloads> HPMS (1).py
223 def search_medical_record():
231         display_medical_record(patient)
232         return
233     elif choice == "name":
234         search_value = input("Enter patient name: ")
235         for patient in patients:
236             if patient["name"] == search_value:
237                 print("ID:", patient["id"])
238                 print("Name:", patient["name"])
239                 display_medical_record(patient)
240                 return
241     else:
242         print("Invalid choice")
243         return
244     print("Patient not found")
245 # Search for a patient
246
247 def search_patient():
248     choice = input("Enter id or name: ")
249     if choice == "id":
250         search_value = input("Enter the patient ID to search: ")
251         for patient in patients:
252             if patient["id"] == search_value:
253                 show_patient(patient)
254                 return
255     elif choice == "name":
256         search_value = input("Enter the patient name to search: ")
257         for patient in patients:
258             if patient["name"] == search_value:
259                 show_patient(patient)
260                 return
261     else:
262         print("Invalid choice")
263         return
264     print("Patient not found")
265
266 def show_patient(patient):
267     print("----- PATIENT FOUND -----")
268     print("ID:", patient["id"])
269     print("Name:", patient["name"])

```

```

266 def show_patient(patient):
267     print("ID:", patient["id"])
268     print("Name:", patient["name"])
269     print("Gender:", patient["gender"])
270     print("Phone:", patient["phone"])
271     print("Address:", patient["address"])
272     print("Age:", patient["age"])
273     print("Disease:", patient["disease"])
274     print("Doctor name:", patient["doctor"])
275     print("Status:", patient["status"])
276     display_medical_record(patient)
277     # Calculate the patient's bill
278
279
280 def recalculate_patient_bill(patient):
281     lab_bill = 0
282     for lab_test in lab_tests:
283         if lab_test["patient_id"] == patient["id"]:
284             lab_bill = lab_bill + lab_test["test_fee"]
285     medicine_charges = 0
286     for medicine in medicines:
287         if medicine["patient_id"] == patient["id"]:
288             medicine_charges = medicine_charges + medicine["total"]
289     pharmacy_bill = medicine_charges
290     doctor_fee = patient["doctor_fee"]
291     administrative_fee = patient["administrative_fee"]
292     other_charges = patient["other_charges"]
293     total = doctor_fee + lab_bill + pharmacy_bill
294     total = total + administrative_fee + other_charges
295     patient["lab_bill"] = lab_bill
296     patient["pharmacy_bill"] = pharmacy_bill
297     patient["medicine_charges"] = medicine_charges
298     patient["bill"] = total
299     return total
300 # Generate a hospital bill
301
302 def generate_bill():
303     choice = input("Enter id or name: ")
304     patient = None
305     if choice == "id":
306         search_value = input("Enter patient ID: ")

```

C: > Users > Lenovo > Downloads > HPMS (1).py

```

302 def generate_bill():
303     if choice == "id":
304         search_value = input("Enter patient ID: ")
305         for saved_patient in patients:
306             if saved_patient["id"] == search_value:
307                 patient = saved_patient
308                 break
309     elif choice == "name":
310         search_value = input("Enter patient name: ")
311         for saved_patient in patients:
312             if saved_patient["name"] == search_value:
313                 patient = saved_patient
314                 break
315     else:
316         print("Invalid choice")
317         return
318     if patient is None:
319         print("Patient not found")
320         return
321     print("----- BILL DETAILS -----")
322     if patient["doctor_fee"] == 0:
323         doctor_fee = input("Enter the doctor fee: ")
324         try:
325             patient["doctor_fee"] = float(doctor_fee)
326         except ValueError:
327             print("Please enter a valid doctor fee")
328             return
329     else:
330         print("Permanent doctor fee:", patient["doctor_fee"])
331     administrative_fee = input("Enter the administrative fee: ")
332     other_charges = input("Enter the other charges: ")
333     try:
334         patient["administrative_fee"] = float(administrative_fee)
335         patient["other_charges"] = float(other_charges)
336     except ValueError:
337         print("Please enter valid amounts")
338         return
339     recalculate_patient_bill(patient)
340     print("----- HOSPITAL BILL -----")
341     print("ID:", patient["id"])

```

```

C: > Users > Lenovo > Downloads > HPMS (1).py
302 def generate_bill():
344     print("Name:", patient["name"])
345     print("Gender:", patient["gender"])
346     print("Phone:", patient["phone"])
347     print("Address:", patient["address"])
348     print("Age:", patient["age"])
349     print("Disease:", patient["disease"])
350     print("Doctor name:", patient["doctor"])
351     print("Doctor fee:", patient["doctor_fee"])
352     print("Lab bill:", patient["lab_bill"])
353     print("Pharmacy bill:", patient["pharmacy_bill"])
354     print("Medicine charges:", patient["medicine_charges"])
355     print("Administrative fee:", patient["administrative_fee"])
356     print("Other charges:", patient["other_charges"])
357     print("Total bill:", patient["bill"])
358     print("Bill generated successfully")
359     # Add medicine charges
360
361 def add_medicine():
362     if len(patients) == 0:
363         print("No patients found")
364         return
365     patient_id = input("Enter patient ID: ")
366     patient = None
367     for saved_patient in patients:
368         if saved_patient["id"] == patient_id:
369             patient = saved_patient
370             break
371     if patient is None:
372         print("Patient not found")
373         return
374     if patient["status"] == "discharged":
375         print("Patient is already discharged")
376         return
377     medicine_name = input("Enter medicine name: ")
378     quantity = input("Enter quantity: ")
379     price = input("Enter price: ")
380     try:
381         quantity = int(quantity)
382         price = float(price)

```

```

C: > Users > Lenovo > Downloads > HPMS (1).py
361 def add_medicine():
381     quantity = int(quantity)
382     price = float(price)
383     except ValueError:
384         print("Please enter valid quantity and price")
385         return
386     total = quantity * price
387     medicine = {
388         "patient_id": patient["id"],
389         "patient_name": patient["name"],
390         "medicine_name": medicine_name,
391         "quantity": quantity,
392         "price": price,
393         "total": total
394     }
395     medicines.append(medicine)
396     recalculate_patient_bill(patient)
397     print("Medicine added successfully")
398     print("Medicine charge:", total)
399     # View medicine details
400
401 def view_medicines():
402     if len(medicines) == 0:
403         print("No medicines found")
404         return
405     print("----- MEDICINES -----")
406     for medicine in medicines:
407         print("Patient ID:", medicine["patient_id"])
408         print("Patient name:", medicine["patient_name"])
409         print("Medicine name:", medicine["medicine_name"])
410         print("Quantity:", medicine["quantity"])
411         print("Price:", medicine["price"])
412         print("Total charge:", medicine["total"])
413         print()
414     # Add a laboratory test
415
416 def add_lab_test():
417     if len(patients) == 0:
418         print("No patients found")
419         return

```

```

416 def add_lab_test():
417     print("No patients found")
418     return
419     patient_id = input("Enter patient ID: ")
420     patient = None
421     for saved_patient in patients:
422         if saved_patient["id"] == patient_id:
423             patient = saved_patient
424             break
425     if patient is None:
426         print("Patient not found")
427         return
428     if patient["status"] == "discharged":
429         print("Patient is already discharged")
430         return
431     test_name = input("Enter test name: ")
432     test_fee = input("Enter test fee: ")
433     test_result = input("Enter test result: ")
434     try:
435         test_fee = float(test_fee)
436     except ValueError:
437         print("Please enter a valid test fee")
438         return
439     lab_test = {
440         "patient_id": patient["id"],
441         "patient_name": patient["name"],
442         "test_name": test_name,
443         "test_fee": test_fee,
444         "test_result": test_result
445     }
446     lab_tests.append(lab_test)
447     recalculate_patient_bill(patient)
448     print("Lab test added successfully")
449 # View laboratory tests
450
451 def view_lab_tests():
452     if len(lab_tests) == 0:
453         print("No lab tests found")
454         return
455     print("----- LABORATORY TESTS -----")
456

```

```

452 def view_lab_tests():
453     return
454     print("LABORATORY TESTS -----")
455     for (function) print: Any
456     print("Patient ID:", lab_test["patient_id"])
457     print("Patient name:", lab_test["patient_name"])
458     print("Test name:", lab_test["test_name"])
459     print("Test fee:", lab_test["test_fee"])
460     print("Test result:", lab_test["test_result"])
461     print()
462 # Discharge a patient
463
464 def discharge_patient():
465     if len(patients) == 0:
466         print("No patients found")
467         return
468     patient_id = input("Enter patient ID to discharge: ")
469     patient = None
470     for saved_patient in patients:
471         if saved_patient["id"] == patient_id:
472             patient = saved_patient
473             break
474     if patient is None:
475         print("Patient not found")
476         return
477     if patient["status"] == "discharged":
478         print("Patient is already discharged")
479         return
480     print("----- FINAL BILL -----")
481     print("ID:", patient["id"])
482     print("Name:", patient["name"])
483     if patient["bill"] > 0:
484         print("Doctor fee:", patient["doctor_fee"])
485         print("Lab bill:", patient["lab_bill"])
486         print("Pharmacy bill:", patient["pharmacy_bill"])
487         print("Administrative fee:", patient["administrative_fee"])
488         print("Other charges:", patient["other_charges"])
489         print("Total bill:", patient["bill"])
490     else:
491         print("Bill has not been generated")

```

```

C: > Users > Lenovo > Downloads > HPMS (1).py
466     def discharge_patient():
494         patient["status"] = "discharged"
495         print("Patient discharged successfully")
496     # Main menu
497     while True:
498         print("... HOSPITAL PATIENT MANAGEMENT SYSTEM... ")
499         print("1. ADD PATIENT")
500         print("2. DISPLAY PATIENT")
501         print("3. ADD MEDICAL RECORDS")
502         print("4. SEARCH MEDICAL RECORD")
503         print("5. SEARCH PATIENT")
504         print("6. GENERATE BILL")
505         print("7. ADD DOCTOR DETAILS")
506         print("8. DISPLAY DOCTOR DETAILS")
507         print("9. BOOK AN APPOINTMENT")
508         print("10. VIEW APPOINTMENTS")
509         print("11. CANCEL APPOINTMENT")
510         print("12. ADD LAB TEST")
511         print("13. VIEW LAB TESTS")
512         print("14. ADD MEDICINE")
513         print("15. VIEW MEDICINES")
514         print("16. DISCHARGE PATIENT")
515         print("17. EXIT")
516         choice = input("Enter your choice: ")
517         if choice == "1":
518             add_patient()
519         elif choice == "2":
520             display_patients()
521         elif choice == "3":
522             add_medical_records()
523         elif choice == "4":
524             search_medical_record()
525         elif choice == "5":
526             search_patient()
527         elif choice == "6":
528             generate_bill()
529         elif choice == "7":
530             add_doctor_details()
531         elif choice == "8":
532             display_doctors()

```

```
522         add_medical_records()
523     elif choice == "4":
524         search_medical_record()
525     elif choice == "5":
526         search_patient()
527     elif choice == "6":
528         generate_bill()
529     elif choice == "7":
530         add_doctor_details()
531     elif choice == "8":
532         display_doctors()
533     elif choice == "9":
534         book_appointment()
535     elif choice == "10":
536         view_appointments()
537     elif choice == "11":
538         cancel_appointment()
539     elif choice == "12":
540         add_lab_test()
541     elif choice == "13":
542         view_lab_tests()
543     elif choice == "14":
544         add_medicine()
545     elif choice == "15":
546         view_medicines()
547     elif choice == "16":
548         discharge_patient()
549     elif choice == "17":
550         print("Thank you")
551         break
552     else:
553         print("Invalid choice. Please enter 1 to 17.")
554
```

... HOSPITAL PATIENT MANAGEMENT SYSTEM...

1. ADD PATIENT
2. DISPLAY PATIENT
3. ADD MEDICAL RECORDS
4. SEARCH MEDICAL RECORD
5. SEARCH PATIENT
6. GENERATE BILL
7. ADD DOCTOR DETAILS
8. DISPLAY DOCTOR DETAILS
9. BOOK AN APPOINTMENT
10. VIEW APPOINTMENTS
11. CANCEL APPOINTMENT
12. ADD LAB TEST
13. VIEW LAB TESTS
14. ADD MEDICINE
15. VIEW MEDICINES
16. DISCHARGE PATIENT
17. EXIT

Enter your choice: 1

Enter patient ID: P101

Enter the name: SHARMA

Enter the gender: MALE

Enter the phone number: 1234567890

Enter the address: BHOPAL

Enter the age: 20

Enter the disease: FEVER

Enter the doctor name: SARUN

Patient added successfully

... HOSPITAL PATIENT MANAGEMENT SYSTEM...

1. ADD PATIENT
2. DISPLAY PATIENT
3. ADD MEDICAL RECORDS
4. SEARCH MEDICAL RECORD
5. SEARCH PATIENT
6. GENERATE BILL
7. ADD DOCTOR DETAILS
8. DISPLAY DOCTOR DETAILS
9. BOOK AN APPOINTMENT
10. VIEW APPOINTMENTS
11. CANCEL APPOINTMENT
12. ADD LAB TEST
13. VIEW LAB TESTS
14. ADD MEDICINE
15. VIEW MEDICINES
16. DISCHARGE PATIENT
17. EXIT

Enter your choice: 2

----- PATIENT DETAILS -----

ID: P101

Name: SHARMA

Gender: MALE

Phone: 1234567890

Address: BHOPAL

Age: 20

Disease: FEVER

Doctor name: SARUN

Status: admitted

Diagnosis:

Symptoms:

Prescription:

Test results:

Treatment details:

... HOSPITAL PATIENT MANAGEMENT SYSTEM...

1. ADD PATIENT
2. DISPLAY PATIENT
3. ADD MEDICAL RECORDS
4. SEARCH MEDICAL RECORD
5. SEARCH PATIENT
6. GENERATE BILL
7. ADD DOCTOR DETAILS
8. DISPLAY DOCTOR DETAILS
9. BOOK AN APPOINTMENT
10. VIEW APPOINTMENTS
11. CANCEL APPOINTMENT
12. ADD LAB TEST
13. VIEW LAB TESTS
14. ADD MEDICINE
15. VIEW MEDICINES
16. DISCHARGE PATIENT
17. EXIT

Enter your choice: 3

Enter id or name: name

Enter the name: SHARMA

Enter diagnosis: ALLERGY

Enter symptoms: MARKS ON SKINS

Enter prescription:

Enter test results: NORMAL

Enter treatment details: BLOOD TEST

Medical record added successfully

```

... HOSPITAL PATIENT MANAGEMENT SYSTEM...
1. ADD PATIENT
2. DISPLAY PATIENT
3. ADD MEDICAL RECORDS
4. SEARCH MEDICAL RECORD
5. SEARCH PATIENT
6. GENERATE BILL
7. ADD DOCTOR DETAILS
8. DISPLAY DOCTOR DETAILS
9. BOOK AN APPOINTMENT
10. VIEW APPOINTMENTS
11. CANCEL APPOINTMENT
12. ADD LAB TEST
13. VIEW LAB TESTS
14. ADD MEDICINE
15. VIEW MEDICINES
16. DISCHARGE PATIENT
17. EXIT
Enter your choice: 4
Enter id or name: name
Enter patient name: SHARMA
ID: P101
Name: SHARMA
Diagnosis: ALLERGY
Symptoms: MARKS ON SKINS
Prescription:
Test results: NORMAL
Treatment details: BLOOD TEST

```

9.1 Patient, Lab, Medicine and Billing Result

```

Patient added successfully
Doctor details added successfully
Lab test added successfully
Medicine added successfully
Medicine charge: 100.0

```

```

----- HOSPITAL BILL -----
ID: P001
Name: Aarav Sharma
Doctor fee: 200.0
Lab bill: 300.0
Pharmacy bill: 100.0
Medicine charges: 100.0
Administrative fee: 100.0
Other charges: 50.0
Total bill: 750.0
Bill generated successfully

```

9.2 Discharge Result

```

----- FINAL BILL -----
ID: P001
Name: Aarav Sharma
Doctor fee: 200.0
Lab bill: 300.0
Pharmacy bill: 100.0
Administrative fee: 100.0
Other charges: 50.0
Total bill: 750.0
Patient discharged successfully

```

Thank you

The execution also verified that the Python file compiles successfully and that the tested workflow can move from patient creation through billing and discharge. The sample data above is test data only and is not intended to represent real patient information.

10. Testing Approach

Testing was approached using feature-based functional tests through the console menu. Each major operation was exercised with valid input, and selected invalid-input and state conditions were considered from the program's implemented checks.

ID	Feature	Input	Expected Result	Observed
T01	Add patient	Unique ID and valid details	Patient added successfully	Pass
T02	Duplicate patient	Existing patient ID	Duplicate ID rejected	Pass
T03	Add doctor	Valid doctor and fee	Doctor added successfully	Pass
T04	Medical record	Existing patient ID	Medical record stored	Pass
T05	Book appointment	Existing patient + doctor	Appointment created with ID	Pass
T06	Add lab test	Existing patient + numeric fee	Lab test stored	Pass
T07	Add medicine	Existing patient + numeric quantity/price	Medicine charge calculated	Pass
T08	Generate bill	Fees and charges entered	Total bill displayed	Pass
T09	Discharge patient	Existing admitted patient	Status changed to discharged	Pass
T10	Exit	Option 17	Program exits with thank-you message	Pass

Additional negative/state checks supported by the code include invalid numeric fee/quantity handling, missing patient/doctor handling, duplicate doctor detection, already-cancelled appointments, and attempts to perform selected operations on discharged patients.

11. Challenges Faced

Managing related records without a database

Patient, doctor, appointment, laboratory, and medicine information must be linked through IDs and names while remaining in separate Python lists. This requires careful searching before updates.

Keeping billing totals consistent

The bill depends on multiple sources of charges. A dedicated `recalculate_patient_bill()` function centralizes the calculation after laboratory and medicine updates.

Handling user input

The application relies on console input, so numeric values need conversion and validation. `try/except` blocks are used for several fee, price, and quantity fields.

Maintaining patient state

The program must distinguish admitted and discharged patients. Several operations check the patient's status before continuing.

Testing an interactive program

Because the application runs through a continuous menu loop, end-to-end testing requires a sequence of menu choices and inputs rather than isolated function calls.

12. Project Summary and Current Limitations

The Hospital Patient Management System demonstrates a complete basic workflow for hospital information management in a console environment. It covers patient registration, medical records, doctors, appointments, laboratory tests, medicines, billing, and discharge through a single Python application.

The current implementation has clear boundaries: records are stored only in memory, there is no authentication, there is no persistent database, and there is no graphical or web interface. These limitations are consistent with the supplied implementation and also identify natural areas for future development if the project is extended.

Potential future extensions

- Persistent database storage for patients, doctors, appointments, medicines, and laboratory tests.
- A graphical or web-based user interface.
- Authentication and role-based access for different hospital staff.
- Date/time validation and richer appointment scheduling.
- Printable/exportable bills and reports.
- Improved input validation and structured error handling.

End of Report